

Federated Adversarial Reentangled Feature Augmentation

Hui Wang¹, Pengcheng Liu⁴, Jie Sun⁵, Renyu Yang^{3*}, Xiaolong Yu¹, Tianyu Wo³, Xudong Liu², Jianwei Niu¹, Chao Tian⁶

¹Hangzhou Innovation Institute, Beihang University, China

²School of Computer Science and Engineering, Beihang University, China

³School of Software, Beihang University, China

⁴Beijing Institute of Astronautical Systems Engineering, China

⁵Zhongguancun Laboratory, China

⁶School of Electronics, Electrical Engineering and Computer Science, Queen’s University Belfast, UK

{whui,renyuyang,xiaolongyu,woty,liuxd,niu Jianwei}@buaa.edu.cn; sunjie@zgclab.edu.cn; c.tian@qub.ac.uk

Abstract—In federated learning, data collected by different clients often contain bias features arising from location, environment, or device differences. Directly using such biased data may lead the model to learn spurious correlations and suffer from model drift. While existing studies attempt to remove bias via feature disentanglement, we observe that bias features also contribute valuable diversity that can enhance model performance when properly leveraged. Based on this insight, we propose FedRAug, a federated feature augmentation technique that re-entangles essential and bias features. Each client trains two encoders to extract disentangled essential and bias representations. Then, two generators are trained to model the distributions of these separated features. Finally, augmented clients created by re-entangling different pairs of generators and incorporated into the global training process. These augmented clients synthesize diverse samples, enriching the training dynamics. Extensive experiments on synthetic and real-world datasets show that FedRAug mitigates the negative impact of biased data and substantially improves overall model performance.

Index Terms—federated learning; feature augmentation.

I. INTRODUCTION

Federated Learning (FL) [1], [2] is a collaborative learning paradigm that enables multiple clients to jointly train a shared model without exposing their private data. It is widely adopted in Internet of Things and edge computing scenarios, where heterogeneous devices such as mobile phones, vehicles, and industrial sensors continuously generate data. However, in such environments, the collected data often exhibits substantial heterogeneity due to variations in device states, environments, or operating conditions, inevitably introducing bias into local datasets (referred to as biased data). For example, images captured by traffic cameras installed in dimly lit tunnels are typically much darker than those collected in well-lit areas. Thus, each dataset contains essential features (e.g., vehicle content) and bias features (e.g., illumination). More generally, a labeled dataset $\mathcal{D}$ consists of pairs $[f, l]$, where the feature vector can be decomposed as $f \triangleq f_e \oplus \varphi f_b$, with φ denoting a fusion coefficient, and the label into essential labels l_e and

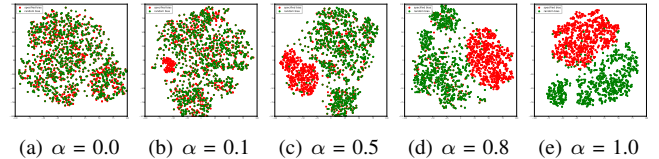


Fig. 1. 2D projection of CMNIST samples with random (green) and consistent (red) bias. Green: $l_e = 2$, l_b random. Red: $l_e = 2$, blue bias for proportion α .

bias labels l_b . For instance, in CMNIST [3], l_e represents the digit identity while l_b corresponds to the color attribute.

The goal of federated training is to learn a correct correlation between f_e and l_e . However, when a stronger or more accessible correlation exists between f_b and l_e , the global model may capture this spurious association, leading to model drift and performance degradation [4]. For example, a traffic camera in a dark region may capture many “dark-background + vehicle” images. A model trained on such biased data may incorrectly learn that a dark background implies the presence of a vehicle, causing misclassification of nighttime pedestrians. To systematically analyze this phenomenon, we introduce controllable spurious correlations between f_b and l_e in the CMNIST, CCIFAR10, and AAFHQ [5] datasets. We partition the training set by l_e to simulate different clients. Within each partition, a proportion $\alpha \in (0, 1]$ of samples share the same l_b label (consistent bias), while the remaining samples are assigned random l_b labels. By varying α , we can precisely control the strength of the spurious correlation and study its influence on model behavior.

Fig. 1 shows a 2D projection of client samples under random and consistent bias. Green points denote samples with random bias, while red points represent those with consistent bias ($l_b = \text{blue}$ in our experiment) applied to a proportion α . When the bias feature f_b is uncorrelated with the label l_e ($\alpha = 0.0$), the two point sets largely overlap. As α increases from 0.1 to 1.0, the points gradually separate and eventually form distinct clusters, indicating that a spurious correlation between f_b and l_e induces a notable distribution shift. Table I shows the model achieves highest accuracy with random bias ($\alpha = 0$). Increasing

* Corresponding author

TABLE I
FEDAVG CLASSIFICATION ACCURACY. “TOTAL”: NUMBER OF CLIENTS,
“CBias”: BIASED CLIENTS, “CBias RATIO”: BIAS PROPORTION.

Total / CBias	CBias Ratio	CMNIST	CCIFAR10	AAFFHQ
100 / 0	0%	90.84 $\pm$ 0.92	47.00 $\pm$ 1.05	60.13 $\pm$ 0.38
100 / 10	10%	89.38 $\pm$ 0.54	46.91 $\pm$ 0.46	59.89 $\pm$ 1.02
	50%	89.03 $\pm$ 0.29	46.72 $\pm$ 0.28	59.81 $\pm$ 0.31
	90%	83.59 $\pm$ 1.01	45.22 $\pm$ 0.31	58.32 $\pm$ 1.11
	100%	80.53 $\pm$ 0.76	40.43 $\pm$ 0.49	58.07 $\pm$ 0.65
100 / 20	10%	90.10 $\pm$ 0.38	46.01 $\pm$ 0.27	59.14 $\pm$ 0.42
	50%	88.78 $\pm$ 0.75	45.21 $\pm$ 0.38	58.73 $\pm$ 1.21
	90%	80.19 $\pm$ 0.29	41.30 $\pm$ 0.83	58.12 $\pm$ 0.82
	100%	70.08 $\pm$ 0.28	40.22 $\pm$ 0.19	58.02 $\pm$ 0.73
100 / 50	10%	89.95 $\pm$ 0.48	45.23 $\pm$ 0.79	58.62 $\pm$ 1.31
	50%	89.47 $\pm$ 0.91	44.81 $\pm$ 0.48	58.23 $\pm$ 0.64
	90%	79.35 $\pm$ 0.27	40.93 $\pm$ 0.59	57.67 $\pm$ 1.14
	100%	57.60 $\pm$ 0.19	33.22 $\pm$ 0.12	57.04 $\pm$ 1.03

the proportion of consistent bias yields progressively lower accuracy, with the degradation further amplified when multiple clients contribute biased samples. In real FL systems, clients such as cameras or sensors may naturally introduce consistent bias due to environmental conditions or device faults, which can adversely affect the global model. Our experiments confirm that such spurious correlations can reduce model accuracy by up to 37.2%. Existing debiasing approaches attempt to suppress misleading bias features but typically rely on prior knowledge, making it difficult to handle diverse and unknown biases in heterogeneous FL data - highlighting the need for alternative solutions.

In this paper, rather than pursuing debiasing, we observe that bias features can positively contribute to data diversity and enhance model performance. Based on this insight, we propose FedRAug, a federated feature augmentation approach leveraging the re-entanglement of essential and bias features. FedRAug employs encoders to disentangle essential and bias features from clients’ data. Feature generators are then trained via adversarial learning to model the distributions of the disentangled features. Finally, generators from different clients are combined to form augmented clients, enabling feature re-entanglement and sample synthesis.

Contributions. (1) We show through preliminary experiments that using biased data from FL clients can greatly degrade federated model performance. (2) We propose a federated augmentation method that disentangles essential and bias features using encoders and conditional GANs, then re-entangles them to increase sample diversity. (3) We evaluate FedRAug on two synthetic datasets (CMNIST and CCIFAR10) and a real-world dataset (AAFFHQ), commonly used in debiasing research [3], [4]. Experiments show that FedRAug consistently outperforms baselines, achieving an average accuracy improvement of 11.9% across the three datasets.

II. RELATED WORK

Data augmentation is commonly used to improve model performance in FL. Existing methods include mixup-based methods, GAN-driven generation, and statistics-guided augmentation. Mixup [6] interpolates samples and labels, while FedMix [7] adapts it to FL using aggregated statistics and

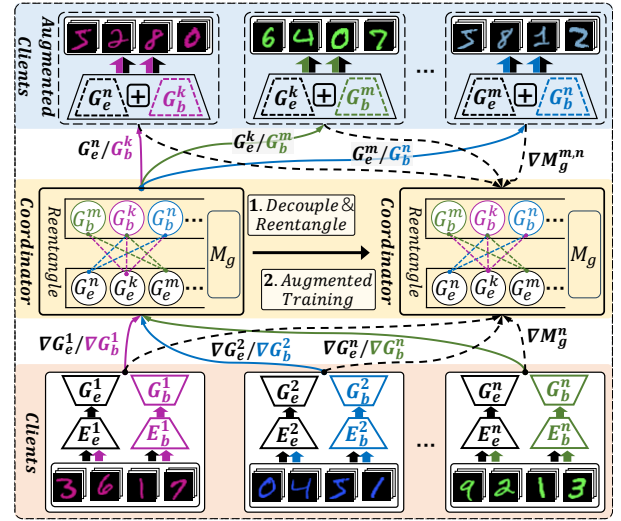


Fig. 2. Architecture of FedRAug. Clients learn disentangled feature distributions via encoders and generators trained adversarially. The coordinator re-entangles selected generators to form augmented clients.

input-gradient approximations. GuidedMixup [8] uses saliency maps to preserve key regions. GAN-based methods, such as FedGAN [9], aggregate local generators to produce synthetic data, while BFFedGAN [10] adds debiasing to reduce distribution shifts in generated samples. Statistics-guided methods operate in the latent space. FedFA [11] perturbs and resamples client feature statistics with respect to global references, whereas FedAvP [12] shares learned augmentation policies instead of raw data. Overall, these methods demonstrate the importance of augmentation in addressing non-IID data and performance challenges in FL.

III. METHODOLOGY

We consider an FL paradigm with a global coordinator and multiple clients (Fig. 2). The coordinator initializes the global model M_g and distributes its weights to clients, which update the model locally and return gradients for aggregation. This process repeats until convergence. FedRAug comprises three stages: (a) clients train two encoders, E_e and E_b , to extract essential and bias features; (b) two generators, G_e and G_b , are trained to model the corresponding feature distributions; (c) generators are re-entangled to form augmented clients that synthesize additional samples to enrich training data. Encoder and generator training strictly follows FL privacy constraints, with clients sharing only model gradients. For M_g , we incorporate a privacy-preserving mechanism via regularization and noise injection during gradient aggregation, and provide theoretical analyses of convergence and privacy guarantees.

A. Integrated Training of Feature Encoders and Generators

In federated training, the goal is to learn a reliable correlation between essential features f_e and labels l_e . However, stronger correlations between bias features f_b and l_e can lead M_g to capture spurious associations. Inspired by disentangled representation learning [4], [13], we train two encoders

$E_e(\cdot; \theta_{ee})$ and $E_b(\cdot; \theta_{eb})$ per client to extract essential features $f_e = E_e(x; \theta_{ee})$ and bias features $f_b = E_b(x; \theta_{eb})$ from local samples. The encoder $E_e(\cdot; \theta_{ee})$ is optimized to minimize the influence of bias features via a cross-entropy (CE) loss:

$$\mathcal{L}_{ee} = \frac{CE(E_b(x; \theta_{eb}), l_e) \cdot CE(E_e(x; \theta_{ee}), l_e)}{CE(E_b(x; \theta_{eb}), l_e) + CE(E_e(x; \theta_{ee}), l_e)}. \quad (1)$$

The loss encourages the encoder $E_e(\cdot; \theta_{ee})$ to focus on essential features, consistent with prior work [3]. In contrast, the encoder $E_b(\cdot; \theta_{eb})$ is optimized using a generalized cross-entropy (GCE) loss [14] defined as follows:

$$\mathcal{L}_{eb} = GCE(E_b(x; \theta_{eb}), l_e). \quad (2)$$

Due to privacy constraints - clients cannot upload their disentangled features to the coordinator - and the limited size of local datasets, we employ conditional generative adversarial networks (CGANs) after the encoders $E_e(\cdot; \theta_{ee})$ and $E_b(\cdot; \theta_{eb})$ to model the distributions of the disentangled features. The encoders project each sample x into the essential and bias feature spaces. The generators $G_e(\cdot; \theta_{ge})$ and $G_b(\cdot; \theta_{gb})$ are trained to approximate the corresponding feature distributions $\mathcal{P}_e$ and $\mathcal{P}_b$. Specifically, $G_e(\cdot; \theta_{ge})$ takes input $ze = [z, l_e]$, concatenating random noise z with the essential feature label l_e to model $\mathcal{P}_e$. Similarly, $G_b(\cdot; \theta_{gb})$ takes input $zb = [z, l_b]$, combining random noise with the bias label l_b to model $\mathcal{P}_b$. The discriminators $D_e(\cdot; \theta_{de})$ and $D_b(\cdot; \theta_{db})$ evaluate the authenticity of generated features by distinguishing them from the encoder outputs $E_e(x; \theta_{ee})$ and $E_b(x; \theta_{eb})$. The generators and discriminators engage in a minimax training procedure through the encoders, as formalized in Eq. 3.

$$\min_G \max_D V(D, G, E) = \mathbb{E}_{\mathbf{x} \sim \mathcal{P}_{data}(x)} [\log D(E(x)|c)] + \mathbb{E}_{\mathbf{z} \sim \mathcal{P}_z(z)} [\log(1 - D(G(z)|c))] , \quad (3)$$

where z denotes random noise, and c corresponds to either l_e or l_b labels. We integrate the encoder E and generator G into a Converged Network (CN), illustrated in Fig. 3. For essential models (E_e, G_e, D_e), the CN optimizes the reconstruction loss $\mathcal{L}_e$ defined in Eq. 4; similarly, bias models (E_b, G_b, D_b) are trained using $\mathcal{L}_b$, with ω denoting the loss fusion coefficient. Following prior work [15], Mean Squared Error (MSE) is used as the loss for both generators and discriminators.

$$\mathcal{L}_e = \mathcal{L}_{ee} + \omega \mathcal{L}_{ge}, \quad \mathcal{L}_b = \mathcal{L}_{eb} + \omega \mathcal{L}_{gb}. \quad (4)$$

B. Distribution Distance between Feature Generators

The coordinator estimates the differences in essential and bias feature distributions, $\mathcal{P}_e$ and $\mathcal{P}_b$, across clients by measuring the distribution distance of their synthetic samples. The synthetic dataset D_g^c from generator G^c (representing either the essential generator G_e^c or the bias generator G_b^c) consists of feature-label pairs: $\{(f^c, l^c) \mid (f^c \in \mathcal{S}_f^e, l^c \in \mathcal{S}_l^e) \text{ or } (f^c \in \mathcal{S}_f^b, l^c \in \mathcal{S}_l^b)\}$, where f^c and l^c denote the generated features and labels, which belong to the feature space $\mathcal{S}_f = \{\mathcal{S}_f^e \cup \mathcal{S}_f^b\}$

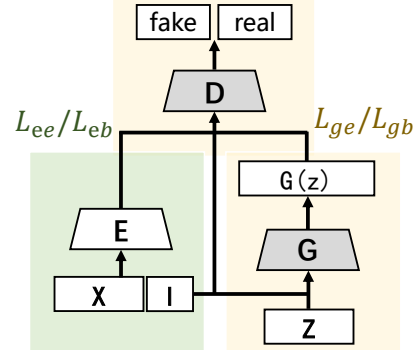


Fig. 3. Converged network. The encoder E extracts decoupled features from input x . Conditioned on label $l = [l_e, l_b]$ and noise z , the generator G synthesizes samples $G(z)$, which are evaluated by the discriminator D alongside real decoupled features.

and label space $\mathcal{S}_l = \{\mathcal{S}_l^e \cup \mathcal{S}_l^b\}$, respectively. The synthetic dataset from client c is written as $D_g^c \triangleq \{(f^{c(i)}, l^{c(i)})\}_{i=1}^{I_c}$, where I_c denotes the number of generated samples. To account for differing contributions of features and labels to distribution discrepancy, we define the distribution-distance vector between synthetic datasets generated by G^m and G^n as follows:

$$\vec{d}(D_g^m, D_g^n) = (d_f(f^m, f^n), d_l(l^m, l^n)) , \quad (5)$$

where $d_f(\cdot, \cdot)$ and $d_l(\cdot, \cdot)$ denote the feature and label distance components. Using the optimal transport (OT) formulation in [16], the distance between two synthetic datasets is:

$$d_{ot}^p = (d_f(f^m, f^n)^p + d_l(l^m, l^n)^p)^{1/p}, \quad p \geq 1, \quad (6)$$

where the feature-wise distance is given by $d_f(f^m, f^n) = \sum_{i=1}^I \|f^{m(i)} - f^{n(i)}\|_2^2$, with I denoting the number of synthetic samples. Since label distances are discrete and not directly comparable, we map them into the feature space via the following transformation:

$$d_l(l^m, l^n) = d_f\left(\frac{1}{I_m} \sum f^m, \frac{1}{I_n} \sum f^n\right), \quad (7)$$

where $f^m \in \mathcal{U}_D(l^m)$, $f^n \in \mathcal{U}_D(l^n)$. We define $\mathcal{U}_D(l^*)$ as the subset of features associated with label l^* , i.e., $\mathcal{U}_D(l^*) \triangleq \{f^* \mid (f^*, l^*) \in D\}$, where $I_m = |\mathcal{U}_D(l^m)|$ and $I_n = |\mathcal{U}_D(l^n)|$. The distribution distance between generators G^m and G^n is:

$$distance(D_g^m, D_g^n) = \|\vec{d}(D_g^m, D_g^n)\|_2 = d_{ot}^2. \quad (8)$$

To quantitatively assess feature distribution discrepancies, we define the **Distribution Difference Mean (DDM)**. Specifically, DDM^i represents the mean distribution distance between client i and the other $C-1$ clients for essential or bias features, where C is the total number of clients.

$$DDM^i = \frac{1}{C-1} \sum_{j=1}^C distance(D_g^i, D_g^j), \quad 1 \leq j \neq i \leq C. \quad (9)$$

Elevated DDM_e^i values indicate that client i exhibits more distinctive essential features, while higher DDM_b^i values reflect

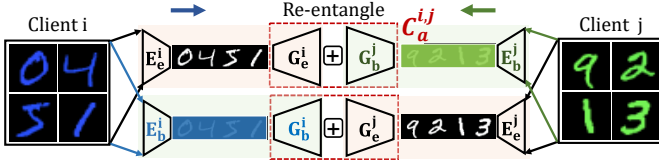


Fig. 4. Feature generator re-entanglement. Essential generator G_e^i and bias generator G_b^j are merged to create the augmented client $C_a^{i,j}$.

greater distinctiveness in bias features. Instances with high metric values - particularly in DDM_e - contribute to improving the diversity of the synthesized samples.

C. Construction of Augmented Clients

Incorporating the augmented client into training enhances the contribution of heterogeneous samples. The formation process is illustrated in Fig. 4. From Eq. 9, we obtain two DDM sequences, DDM_e and DDM_b , corresponding to essential and bias feature generators of all clients. Generators appearing earlier in DDM_e produce sparser essential features that are beneficial for model performance, while those ranked earlier in DDM_b exhibit more pronounced bias (e.g., extreme lighting in traffic-camera data). The augmented client is formed through cross-entanglement of generators selected from the two DDM sequences. The entanglement strategy follows two principles: **(a)** Generators producing sparser essential features are prioritized to improve model performance; thus, selection proceeds according to the order in DDM_e . **(b)** Because a client's state or environment typically changes gradually, each selected essential generator is fused with bias generators within a fusion window in DDM_b . This window is anchored by the bias generator corresponding to the same client as the essential generator. Only the top-ranked essential generators - referred to as source generators (SG) - participate in fusion. Fig. 5 illustrates the generator re-entanglement matching strategy, which is formally defined in Eq. 10.

$$C_a^{i,j} = \lambda G_e^i + (1 - \lambda) G_b^j, \quad l_a^{i,j} = [l_e^i, l_b^j], \quad \lambda \in (0, 1], \quad (10)$$

where $C_a^{i,j}$ denotes the augmented client constructed by combining the essential generator G_e^i of client i and the bias generator G_b^j of client j . The combined label $l_a^{i,j}$ is formed by pairing the essential label l_e^i with the bias label l_b^j . The coefficient λ is a hyperparameter controlling the contribution of the feature generator. When $\lambda = 1$, the augmented samples rely on the essential features from G_e^i , effectively removing bias. In contrast, $\lambda = 0$ causes the samples to inherit only bias features from G_b^j , which offers limited training benefit.

D. Theoretical Analysis of the Feature Generators

We analyze the convergence of the feature generators and their privacy-preserving properties.

Proposition 1. (Convergence). *For client c , the optimized local generators G_e^c and G_b^c converge to unique solutions G_e^{c*} and G_b^{c*} , which approximate the essential and bias feature distributions, i.e., $G_e^{c*} \rightarrow \mathcal{P}_e^c(x)$, $G_b^{c*} \rightarrow \mathcal{P}_b^c(x)$.*

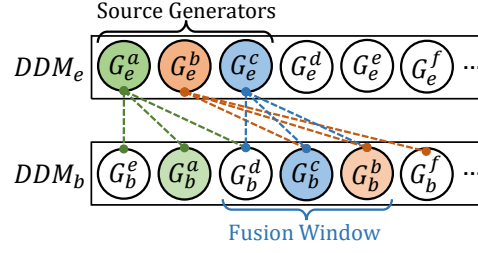


Fig. 5. Generator re-entanglement matching strategy.

Proposition 2. (Privacy Guarantee). *For client c , any generator G that does not obtain the weights or architecture of G_e^c at initialization or during any communication round cannot reconstruct the underlying feature distribution $\mathcal{P}_e^c(x)$.*

E. Privacy-preserving training of the federated model M_g

To protect the privacy of both original and augmented clients during the training of M_g , following [17], we refine the FedAvg algorithm [1] in each optimization round as follows: (1) sample clients (original or augmented) with probability q , (2) clip each client's gradient to a bounded norm, and (3) add Gaussian noise. The gradient update of M_g at round t is then:

$$\begin{aligned} \nabla \theta^{t+1} &\leftarrow \nabla \theta^t + \frac{\sum \omega_c \Psi(\theta_c^t)}{q \sum \omega_c} + \mathcal{N}\left(0, I\left(\frac{20\mathcal{Z}}{q \sum \omega_c}\right)^2\right), \text{ where} \\ \Psi(\theta_c^t) &= \text{Clip}\left(\frac{1}{\mathcal{B}_c} \sum_{x \in \mathcal{B}_c} \nabla l(x; \theta_c^t)\right), \quad \text{Clip}(v) = v \cdot \min\left(1, \frac{20}{\|v\|_2}\right), \end{aligned} \quad (11)$$

where $q \in (0, 1]$ is the client selection probability, and c denotes the selected clients. θ^t represents the weights of M_g , and θ_c^t is the local copy on client c . The weight factor $\omega_c = \min(n_c/n, 1)$, where n_c is the number of samples of client c , and n prevents clients with large datasets from dominating updates. $\mathcal{Z}$ is the noise scale, and $\mathcal{N}$ follows the normal distribution. $\mathcal{B}_c$ is the batch size, and $l(\cdot)$ is the loss function. Privacy in Eq. 11 is achieved through gradient clipping and Gaussian noise injection, following the standard DP-FedAvg framework [17].

IV. EXPERIMENTS

Datasets. CMNIST: The CMNIST dataset is constructed by introducing diverse background colors into MNIST [18]. Each sample is assigned a combined label [*digit, color*] to denote essential and bias features. CCIFAR10: The Corrupted CIFAR-10 dataset (CCIFAR10) is created by adding various textures to CIFAR-10, following the design protocol of Hendrycks [19]. Each sample has a combined label [*class, texture*]. AAFFHQ: The Attr-Aligned FFHQ dataset (AAFFHQ) is obtained by regrouping FFHQ [5] samples. Gender serves as the essential feature, and hair color as the bias feature.

Baselines. Our baselines include Vanilla training, Guided-Mixup [8], debiasing-based approaches (DRDAug [4], Bi-aSwap [20]), and FL methods (FedAvg [1], FedMix [7], FedFA [11], FedAvP [12]). The vanilla baseline trains a centralized model on the union of client data without any augmentation

TABLE II

COMPARISON OF ACCURACY. BC DENOTES THE PROPORTION OF BIASED CLIENTS AND THE PROPORTION OF SAMPLES EXHIBITING CONSISTENT BIAS.

Dataset	BC	Vanilla	GuidedMixup	DRDAug	BiaSwap	FedAvg	FedMix	FedFA	FedAvP	FedRAug
CMNIST	0.0+0.0	82.34 $\pm$ 0.38	91.46 $\pm$ 0.61	89.74 $\pm$ 1.45	90.68 $\pm$ 0.73	90.84 $\pm$ 0.92	91.75 $\pm$ 0.84	91.87 $\pm$ 1.02	87.65 $\pm$ 0.44	94.84 $\pm$ 0.98
	0.1+0.5	83.03 $\pm$ 1.83	91.28 $\pm$ 1.03	89.45 $\pm$ 2.01	88.37 $\pm$ 1.42	89.03 $\pm$ 0.29	91.20 $\pm$ 0.71	91.42 $\pm$ 0.81	86.93 $\pm$ 0.12	94.03 $\pm$ 0.48
	0.1+0.7	80.67 $\pm$ 0.84	90.63 $\pm$ 0.64	84.39 $\pm$ 0.89	86.54 $\pm$ 2.71	84.31 $\pm$ 0.65	90.84 $\pm$ 1.84	90.74 $\pm$ 1.41	85.64 $\pm$ 0.52	94.59 $\pm$ 1.01
	0.5+0.5	76.94 $\pm$ 1.77	89.93 $\pm$ 1.13	83.29 $\pm$ 1.87	85.67 $\pm$ 1.92	88.78 $\pm$ 0.75	90.02 $\pm$ 0.77	90.62 $\pm$ 0.83	84.25 $\pm$ 0.55	93.78 $\pm$ 0.73
	0.7+0.7	70.49 $\pm$ 0.36	88.97 $\pm$ 0.32	81.24 $\pm$ 2.98	84.72 $\pm$ 2.72	80.34 $\pm$ 0.85	88.63 $\pm$ 1.83	89.82 $\pm$ 0.84	83.21 $\pm$ 0.24	94.47 $\pm$ 0.91
CCIFAR10	0.0+0.0	41.48 $\pm$ 0.75	52.93 $\pm$ 0.51	52.49 $\pm$ 1.58	48.92 $\pm$ 1.15	44.00 $\pm$ 1.05	52.84 $\pm$ 0.28	53.02 $\pm$ 0.87	52.03 $\pm$ 0.53	53.46 $\pm$ 1.15
	0.1+0.5	40.23 $\pm$ 0.21	52.47 $\pm$ 2.21	44.30 $\pm$ 1.84	46.21 $\pm$ 0.46	43.72 $\pm$ 0.28	52.01 $\pm$ 0.81	52.54 $\pm$ 0.74	51.45 $\pm$ 0.41	53.52 $\pm$ 0.38
	0.1+0.7	40.86 $\pm$ 0.12	51.63 $\pm$ 0.24	44.55 $\pm$ 0.98	46.42 $\pm$ 1.56	43.22 $\pm$ 0.31	51.01 $\pm$ 1.01	51.64 $\pm$ 1.02	51.04 $\pm$ 0.02	52.26 $\pm$ 0.71
	0.5+0.5	36.38 $\pm$ 0.19	50.82 $\pm$ 1.21	42.48 $\pm$ 2.01	43.82 $\pm$ 0.78	42.21 $\pm$ 0.38	50.34 $\pm$ 0.71	51.32 $\pm$ 0.18	50.64 $\pm$ 0.12	52.81 $\pm$ 0.72
	0.7+0.7	32.38 $\pm$ 1.87	50.22 $\pm$ 0.64	39.46 $\pm$ 1.28	41.74 $\pm$ 0.64	38.04 $\pm$ 1.03	48.84 $\pm$ 0.78	50.71 $\pm$ 1.89	49.89 $\pm$ 0.31	52.93 $\pm$ 0.59
AAFFHQ	0.0+0.0	58.38 $\pm$ 1.94	63.72 $\pm$ 1.43	62.84 $\pm$ 0.94	60.29 $\pm$ 0.24	59.73 $\pm$ 0.87	63.84 $\pm$ 0.76	63.97 $\pm$ 2.01	62.42 $\pm$ 0.52	65.62 $\pm$ 1.03
	0.1+0.5	58.29 $\pm$ 0.84	64.03 $\pm$ 1.72	62.03 $\pm$ 1.05	59.09 $\pm$ 0.65	59.14 $\pm$ 0.47	63.41 $\pm$ 0.92	64.32 $\pm$ 1.02	62.25 $\pm$ 0.62	65.34 $\pm$ 0.74
	0.1+0.7	56.48 $\pm$ 0.94	63.20 $\pm$ 1.27	61.83 $\pm$ 0.84	58.71 $\pm$ 1.58	56.47 $\pm$ 0.38	62.52 $\pm$ 1.63	63.78 $\pm$ 0.61	61.89 $\pm$ 0.41	65.23 $\pm$ 0.75
	0.5+0.5	54.94 $\pm$ 0.75	62.73 $\pm$ 0.51	61.02 $\pm$ 0.57	58.19 $\pm$ 1.28	55.74 $\pm$ 0.27	61.94 $\pm$ 0.74	63.51 $\pm$ 0.51	61.24 $\pm$ 0.41	64.87 $\pm$ 0.74
	0.7+0.7	55.01 $\pm$ 0.75	61.65 $\pm$ 1.23	59.74 $\pm$ 0.83	57.61 $\pm$ 1.32	55.84 $\pm$ 0.72	60.83 $\pm$ 0.61	62.78 $\pm$ 0.98	59.53 $\pm$ 0.51	64.95 $\pm$ 0.92

or debiasing. GuidedMixup performs saliency-aware mixup to generate informative samples. DRDAug learns debiased representations by suppressing bias features, while BiaSwap performs bias-specific augmentation by exchanging bias attributes across samples. FedAvg aggregates model updates from distributed clients to train a global model. FedMix conducts mixup-based data augmentation in a federated manner. FedFA introduces probabilistic feature-level augmentation using client feature statistics. FedAvP improves model performance by sharing learned augmentation policies instead of raw data.

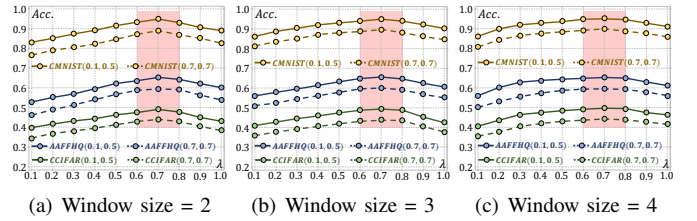
Implementation Details. We employ a convolutional neural network (CNN) and a five-layer perceptron for the federated model M_g and feature encoders. Each encoder maps a one-dimensional input vector to a feature vector of the same size. For feature generators G_e and G_b , we adopt a Conditional GAN (CGAN) with conditioning variable $c = [z, l]$, where z is 100-dimensional random noise and l is a one-hot encoded label. During training, M_g is optimized using cross-entropy loss with a learning rate of 10^{-3} . In the gradient update (Eq.11), the client selection probability q , sample size upper bound n , noise scale $\mathcal{Z}$, and batch size $\mathcal{B}_c$ are set to 0.9, 10^4 , 10^2 , and 64, respectively. The decoupling epoch T_{dec} and augmentation epoch T_{aug} are both 10^3 . The loss fusion factor ω is 0.5. The number of clients is $C = 100$, with source generators $SG = 0.2C$. The re-entanglement coefficient λ is 0.7. Each augmented client produces 10^3 synthetic samples, and the fusion window size $WS = 3$.

A. Quantitative Evaluation

Comparison of Accuracy. Table II summarizes the accuracy results. Overall, FedRAug delivers the best performance across all datasets and notably stabilizes model behavior under various bias conditions, achieving the lowest accuracy variance. Compared with debiasing baselines, FedRAug consistently yields substantial gains, demonstrating that cross-client feature coupling provides a more effective augmentation strategy for robust FL. FedRAug also outperforms FedMix and FedFA: the former applies sample-level mixup, while the latter perturbs client-specific feature statistics using normal distributions - both overlook interference from biased clients. Moreover,

FedRAug surpasses FedAvg by 9.2%, 23.5%, and 14.2% on the three datasets, benefiting from the increased sample diversity introduced through feature re-entanglement.

Fusion Performance. Fig. 6 illustrates the effect of the fusion coefficient λ on model performance. Accuracy rises as λ increases from 0 to 0.7, peaking around 0.7 (red regions), and then decreases for $\lambda > 0.7$. Small λ values produce augmented samples dominated by bias and lacking essential features, sustaining spurious correlations and lowering accuracy. As λ grows, essential features dominate, mitigating bias effects. A larger fusion window accelerates the attainment of peak accuracy by enhancing feature diversity, but exceeding the optimal point increases computation without further benefit.

Fig. 6. Effect of fusion coefficient λ on model accuracy.

Diversity of Generated Samples. Fig. 7 visualizes the diversity of augmented samples via 2D feature projection. The augmented client combines generators G_e and G_b , learned from two clients' data. The separated distributions of G_e (blue) and G_b (orange) illustrate the distinction between essential and bias features. The augmented samples (red) span a broader region than the original client samples (green), indicating diversity. A small fusion coefficient (e.g., $\lambda = 0.3$) overemphasizes bias features, whereas a large one (e.g., $\lambda = 0.9$) amplifies essential features, both of which may hinder generalization.

Training Strategies. Table III shows that the timing of augmented client inclusion minimally impacts model performance. The strategies are: 1) $C \rightarrow AC$: clients train the initial model, followed by augmented clients on the intermediate model. 2) $AC \rightarrow C$: the reverse order. 3) $C + AC$: clients and augmented clients train simultaneously. Accuracy is comparable across strategies with low variance. To reduce complexity, our algorithm trains clients during feature decoupling,

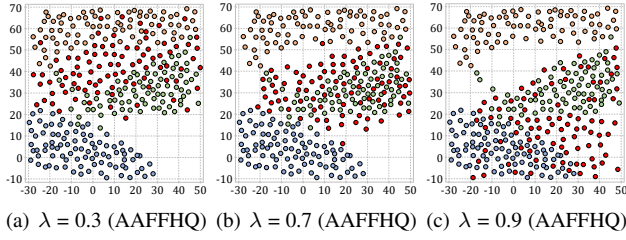


Fig. 7. 2D projection of samples. Green: original samples. Blue/Orange: essential/bias features. Red: augmented client samples.

with augmented clients continuing on the intermediate model, without significantly affecting performance.

TABLE III
COMPARISON OF ACCURACY UNDER DIFFERENT TRAINING STRATEGIES.

BC	Strategy	CMNIST	CCIFAR10	AAFFHQ
0.1 + 0.7	C → AC	94.59 ± 1.01	52.26 ± 0.71	65.23 ± 0.75
	AC → C	94.67 ± 0.45	53.01 ± 0.27	64.76 ± 0.19
	C + AC	94.87 ± 1.02	52.89 ± 1.34	64.98 ± 1.04
0.5 + 0.7	C → AC	93.19 ± 1.08	52.30 ± 0.83	64.76 ± 0.83
	AC → C	93.87 ± 0.39	53.03 ± 1.05	65.12 ± 0.36
	C + AC	94.14 ± 0.83	52.97 ± 1.03	65.32 ± 0.29
0.7 + 0.7	C → AC	94.47 ± 0.91	52.93 ± 0.59	64.95 ± 0.92
	AC → C	95.01 ± 1.04	53.86 ± 1.93	65.21 ± 0.21
	C + AC	94.38 ± 0.81	53.82 ± 0.54	65.43 ± 0.71

Training Cost. The additional Computational Cost of FedRAug mainly comes from (1) training the feature generators and (2) augmented training of the model M_g . All computation occurs during offline training. We estimate per-sample FLOPs of M_g and generators as U_m and U_g . The total computation introduced by FedRAug is $Total = 2CPU_g + SG \cdot WS \cdot AS \cdot U_m$, where C is the number of clients, P is the per-client sample size, SG is the number of source generators, WS is the fusion window size, and AS is the number of synthetic samples per augmented client. The computational growth rate is $CGR(\%) = Total / (CPU_m)$. FedRAug introduces only a moderate increase in Communication Cost. Since all augmented clients are co-located with the coordinator, no communication occurs among them, and the number of communication rounds remains unchanged. In each round, only gradients of M_g and generators are transmitted. The communication costs for training original samples, generators, and synthetic samples are $2CPP_m/B$, $4CPP_g/B$, and $2P_mSG \cdot WS \cdot AS/B$, where P_m and P_g denote parameter sizes of M_g and the generator, and B is the batch size. The communication growth rate is $MGR(\%) = \frac{4CPP_g/B + 2P_mSG \cdot WS \cdot AS/B}{2CPP_m/B} = \frac{2CPP_g + P_mSG \cdot WS \cdot AS}{CPP_m}$. Taking $C=100$, $SG=20$, and $AS=100$ as an example, Table IV reports the computational cost, CGR , and MGR of FedRAug. With fusion windows of 2 and 3, the CGR and MGR are 4.3%/7.9% and 7.2%/9.6%, respectively. Although FedRAug adds computation and communication overhead during training, it achieves the best performance and stability under bias conditions.

V. CONCLUSIONS

This work investigates the impact of bias features in FL and proposes a decoupled feature re-entanglement method.

TABLE IV
EVALUATION OF COMPUTATIONAL COST (GFLOPs), CGR (%), AND MGR (%). $TGS = SG \cdot WS \cdot AS$: TOTAL GENERATED SAMPLES.

WS	TGS	CMNIST			CCIFAR10			AAFFHQ		
		Cost	CGR	MGR	Cost	CGR	MGR	Cost	CGR	MGR
2	04K	3.48	04.4	07.5	2.94	04.9	07.3	2.06	03.7	06.6
3	06K	5.04	07.8	09.8	4.26	08.9	09.3	2.88	07.1	09.7
4	08K	6.60	11.1	12.1	5.58	12.9	12.4	3.70	10.9	12.1
5	10K	8.16	14.4	14.5	6.91	14.8	13.9	4.52	13.7	14.6

By disentangling essential and bias features with separate encoders and generating augmented clients via CGANs, the approach effectively enhances model performance. Extensive experiments demonstrate its efficacy.

VI. ACKNOWLEDGMENTS

This work is supported in part by National Key R&D Program of China (Grant No. 2024YFB4505901), the National Natural Science Foundation of China (Grant No. 62402024), the Beijing Natural Science Foundation (Grant No. L241050), the Zhejiang Province Key R&D Program (2023C01070), and the Fundamental Research Funds for the Central Universities.

REFERENCES

- [1] Brendan McMahan et al., “Communication-efficient learning of deep networks from decentralized data,” PMLR, 2017, pp. 1273–1282.
- [2] Hui Wang, Jie Sun, Tianyu Wo, and Xudong Liu, “Fed-3da: a dynamic and personalized federated learning framework,” in *ICASSP*, 2023.
- [3] Junhyun Nam et al., “Learning from failure: De-biasing classifier from biased classifier,” *NIPS*, vol. 33, pp. 20673–20684, 2020.
- [4] Jungsoo Lee et al., “Learning debiased representation via disentangled feature augmentation,” *NIPS*, vol. 34, pp. 25123–25133, 2021.
- [5] Tero Karras, Samuli Laine, and Timo Aila, “A style-based generator architecture for generative adversarial networks,” in *CVPR*, 2019.
- [6] Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz, “mixup: Beyond empirical risk minimization,” in *ICLR*, 2018.
- [7] Tehrim Yoon et al., “Fedmix: Approximation of mixup under mean augmented federated learning,” in *ICLR*, 2021.
- [8] Minsoo Kang and Suhyun Kim, “Guidedmixup: An efficient mixup strategy guided by saliency maps,” *AAAI*, vol. 37, no. 1, Jun. 2023.
- [9] Song Han et al., “Fed-gan: Federated gan with privacy-preserving for cross-device scenarios,” *IEEE TDS*, vol. 22, no. 5, 2025.
- [10] Vaikkunth Mugunthan et al., “Bias-free fedgan: A federated approach to generate bias-free datasets,” *arXiv preprint arXiv:2103.09876*, 2021.
- [11] Tianfei Zhou, “Fedfa: Federated feature augmentation,” International Conference on Learning Representations (ICLR), 2023.
- [12] Minui Hong, Junhyeog Yun, Insu Jeon, et al., “Fedavp: augment local data via shared policy in federated learning,” 2024, NeurIPS.
- [13] Hui Wang, Jie Sun, Tianyu Wo, and Xudong Liu, “Fedfrr: Federated forgetting-resistant representation learning,” in *ICME*, 2024, pp. 1–6.
- [14] Zhilu Zhang and Mert Sabuncu, “Generalized cross entropy loss for training deep neural networks with noisy labels,” *NIPS*, vol. 31, 2018.
- [15] Xi Chen et al., “Infogan: Interpretable representation learning by information maximizing gans,” *NIPS*, vol. 29, 2016.
- [16] David Alvarez Melis and Nicolo Fusi, “Geometric dataset distances via optimal transport,” *NIPS*, vol. 33, 2020.
- [17] H Brendan McMahan et al., “Learning differentially private recurrent language models,” in *ICLR*, 2018.
- [18] Yann LeCun et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, pp. 2278–2324, 1998.
- [19] Dan Hendrycks and Thomas Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” in *ICLR*, 2018.
- [20] E. Kim and J. Lee, “Biaswap: Removing dataset bias with bias-tailored swapping augmentation,” in *IEEE/CVF ICCV*, 2021, pp. 14972–14981.